Rapport de projet

Filière : IID1 | Module : Développement Web et Mobile

RevisionFlow

Planificateur de révisions intelligent

Encadrante : Pr. RABHI Loubna

Réalisé par : NOUGBOLO Godwin Elie

Année universitaire : 2025 – 2026

Table des matières

1 Introduction et problème résolu **3**
1.1 Contexte étudiant . . . 3
1.2 Solution proposée . . . 3

2 Présentation générale de l'application **3**
2.1 La page d'accueil (landing page) . . . 3
2.2 L'assistant de création (wizard) . . . 4
2.3 Le tableau de bord (dashboard) . . . 4
2.4 L'historique . . . 4

3 Fonctionnalités détaillées **4**
3.1 Assistant de création en 3 étapes . . . 4
3.1.1 Étape 1 : Profil . . . 4
3.1.2 Étape 2 : Modules . . . 4
3.1.3 Étape 3 : Disponibilités . . . 5
3.2 Tableau de bord quotidien . . . 5
3.2.1 Vue « Aujourd'hui » . . . 5
3.2.2 Actions disponibles sur une session . . . 5
3.2.3 Système de filtres combinables . . . 5
3.2.4 Minuteur Pomodoro intégré . . . 6
3.3 Vue calendrier mensuelle . . . 6
3.4 Gestion des jours fériés . . . 6
3.5 Persistance et échange de données . . . 7
3.6 Mode sombre . . . 7

4 Algorithmes utilisés **7**
4.1 Formule de priorité dynamique . . . 7
4.2 Entrelacement des sessions . . . 8
4.3 Algorithme glouton de distribution . . . 8
4.3.1 Capacités selon le type de jour . . . 8
4.4 Respect du passé et recalcul du futur . . . 9
4.5 Gestion du backlog et transparence . . . 9

5 Architecture technique **9**
5.1 Technologies utilisées . . . 10
5.2 Structure du projet . . . 10
5.3 Gestion de l'état — pattern Store . . . 10
5.4 Design system . . . 11

6 Défis techniques rencontrés **11**
6.1 Préserver le passé tout en recalculant le futur . . . 11
6.2 Synchronisation Pomodoro et planning . . . 11
6.3 Tri déterministe de l'algorithme . . . 11
6.4 Migration des données sauvegardées . . . 12

7 Interface utilisateur et design **12**
7.1 Philosophie de design . . . 12

7.2 Typographie 12
7.3 Responsive et mobile 12

8 Perspectives d'amélioration 12
8.1 Améliorations fonctionnelles 12
8.2 Améliorations techniques 13
8.3 Évolutions vers le mobile 13

9 Conclusion 13

1. Introduction et problème résolu

1.1. Contexte étudiant

Les étudiants en cycle d'ingénieur doivent gérer plusieurs examens en même temps, avec des dates butoir différentes et des niveaux de difficulté variables. En pratique, beaucoup utilisent un agenda papier ou un simple tableur, mais ces outils présentent deux limites importantes :

— **Rigidité face aux imprévus :** si un examen change de date, il faut tout recalculer manuellement, ce qui prend du temps et décourage.
— **Absence de signal d'alerte :** aucun outil classique ne détecte qu'un planning est irréaliste (trop de chapitres, pas assez de jours).

En résulte souvent un phénomène bien connu : la *panique de dernière minute*, où l'étudiant réalise trop tard qu'il n'a pas le temps de tout réviser.

1.2. Solution proposée

RevisionFlow est une application web qui génère automatiquement un planning de révisions, le met à jour en temps réel après chaque action de l'étudiant, et affiche clairement les sessions impossibles à placer.

Fonctionnement

L'application fonctionne entièrement dans le navigateur, sans inscription ni serveur. Toutes les données restent stockées localement sur l'ordinateur de l'étudiant, ce qui garantit la confidentialité et la disponibilité hors connexion.

Les trois principes fondateurs sont :

1. **Zéro friction :** aucun compte à créer, aucune installation.
2. **Transparence :** si le planning est irréaliste, l'application le dit.
3. **Réactivité :** toute modification déclenche un recalcul immédiat.

2. Présentation générale de l'application

RevisionFlow est organisée en quatre interfaces principales, chacune correspondant à une étape du parcours de l'étudiant.

2.1. La page d'accueil (landing page)

La landing page est la première chose que l'étudiant voit. Elle présente l'application de façon claire et moderne, avec trois sections principales :

— **Héros :** un titre accrocheur, un sous-titre explicatif, et un bouton « Créer mon planning ». En dessous, un mini-tableau de bord animé donne un aperçu concret de l'interface finale.
— **Problème :** trois cartes décrivent les frustrations classiques des étudiants (incertitude, imprévus, surcharge).
— **Solution et méthode :** présentation des fonctionnalités clés en trois étapes.

Concept clé

La landing page intègre des effets visuels avancés entièrement en JavaScript pur : **parallaxe au scroll** (les cercles de fond bougent à des vitesses différentes), **flou progressif** du mini-dashboard quand l'utilisateur scrolle, **changement de couleur de la navbar** selon la section visible, et des compteurs animés qui s'activent quand le dashboard entre dans le champ de vision.

2.2. L'assistant de création (wizard)

Au premier lancement, l'étudiant passe par un assistant en 3 étapes pour configurer son planning initial. Cette interface est conçue pour être rapide et guidée.

2.3. Le tableau de bord (dashboard)

C'est l'interface principale, utilisée au quotidien. Elle regroupe quatre onglets : Aujourd'hui, Calendrier, Modules, et Réglages.

2.4. L'historique

Une page séparée affiche les 5 derniers plannings archivés, en lecture seule, avec les statistiques associées (taux de complétion, streak, vélocité).

3. Fonctionnalités détaillées

3.1. Assistant de création en 3 étapes

3.1.1. Étape 1 : Profil

L'étudiant configure trois informations :

1. **Type d'organisation :** Fixe ou Flexible.
2. **Pays :** pour récupérer automatiquement les jours fériés via l'API Nager.Date.
3. **Date de début :** à partir de quand générer le planning.

Concept clé

Profil Fixe vs Flexible
Les deux profils utilisent exactement le même algorithme de planification. La différence est avant tout dans l'usage quotidien :

- **Fixe** (concours, examens nationaux) : les dates sont imposées et stables.
- **Flexible** (contrôle continu, projets) : les dates peuvent changer à tout moment. La modification d'une date d'examen est très accessible et déclenche immédiatement un recalcul complet du planning futur.

3.1.2. Étape 2 : Modules

Pour chaque matière, l'étudiant renseigne :

- **Nom** du module (ex. : Analyse, Algorithmique)
- **Date d'examen**
- **Difficulté** : de 1 à 5 étoiles

— **Nombre de chapitres** : estimation du volume de travail

Un indicateur de priorité (Haute, Moyenne, Basse) est calculé en temps réel au fur et à mesure que l'étudiant remplit les champs.

3.1.3. Étape 3 : Disponibilités

L'étudiant configure son rythme de travail :

— Nombre de sessions le soir en semaine (par défaut : 2h)
— Nombre de sessions le week-end (par défaut : 6h)
— Durée d'une session par chapitre (par défaut : 1h)

Un calendrier interactif permet de bloquer manuellement des jours de repos.

3.2. Tableau de bord quotidien

3.2.1. Vue « Aujourd'hui »

C'est l'onglet principal, affiché par défaut à chaque ouverture. Il montre :

— Une **barre de statistiques** : sessions faites/total du jour, progression globale, streak, jours avant le prochain examen.
— Un **anneau de progression** du jour (en pourcentage).
— La **liste des sessions** à effectuer, triées intelligemment.
— Un aperçu des **5 prochains jours** avec les sessions planifiées.

3.2.2. Actions disponibles sur une session

Action	Effet sur le planning
Valider ()	La session est marquée comme faite. Le compteur du module augmente. Le planning futur se recalcule.
Reporter ()	La session est retirée du jour actuel et replacée automatiquement dans les jours futurs selon la priorité.
Anticiper ()	Une session du lendemain est avancée à aujourd'hui, si la capacité du jour le permet.
Jour libre (+)	Transforme la journée en jour à haute capacité pour rattraper un retard.

3.2.3. Système de filtres combinables

Une fonctionnalité avancée permet de filtrer les sessions selon trois critères simultanément :

— **Statut :** Tous / À faire / Faits / Manqués
— **Module :** Tous / un module spécifique
— **Semaine :** Toutes / une semaine précise (avec son taux de complétion)

Les filtres actifs sont persistés dans l'état de l'application entre les sessions.

3.2.4. Minuteur Pomodoro intégré

Le Pomodoro est un composant autonome, toujours visible dans la barre latérale. Il fonctionne selon la méthode classique : 25 minutes de travail, 5 minutes de pause courte, 15 minutes de pause longue après 4 sessions.

À la fin de chaque session, une **notification sonore** est émise via l'API Web Audio, et une fenêtre de confirmation propose de valider la session correspondante dans le planning.

3.3. Vue calendrier mensuelle

Un calendrier mensuel avec un code couleur clair pour chaque jour :

Couleur	Signification	Condition
Vert foncé	Journée complète	Toutes les sessions du jour sont validées
Vert clair	En cours	Des sessions restent à faire (jour futur)
Rouge clair	Sessions manquées	Des sessions non faites sur un jour passé
Gris	Repos	Aucune session planifiée ce jour-là
Bordure orange	Examen	Date d'examen d'un module

3.4. Gestion des jours fériés

L'application interroge l'API publique **Nager.Date** pour récupérer les jours fériés du pays sélectionné.

Fonctionnement

Exemple de réponse de l'API Nager.Date (Maroc, 2025) :

```
GET https://date.nager.at/api/v3/PublicHolidays/2025/MA

[
  { "date": "2025-01-01", "localName": "Jour de l'An" },
  { "date": "2025-01-11", "localName": "Manifeste de l'
     Independance" },
  { "date": "2025-05-01", "localName": "Fete du Travail" }
]
```

L'application récupère les jours fériés de l'année en cours et de l'année suivante en parallèle via `Promise.all`, puis les traite comme des jours à haute capacité, identique aux week-ends.

3.5. Persistance et échange de données

Mécanisme	Description
Sauvegarde automatique	À chaque action, l'état complet est sérialisé en JSON et stocké dans le `localStorage`.
Restauration au démarrage	Si une sauvegarde existe, elle est chargée et le planning est recalculé pour intégrer les jours écoulés.
Export JSON	Téléchargement du planning complet en fichier `.json` nommé automatiquement avec la date du jour.
Import JSON	Restauration depuis un fichier `.json` avec validation de structure avant l'import.
Historique	Les 5 derniers plannings archivés sont conservés en lecture seule, avec taux de complétion, streak, vélocité et détail par module.

3.6. Mode sombre

L'application intègre un thème sombre complet, activable depuis les réglages en un clic. Toutes les variables CSS sont redéfinies dans un bloc `[data-theme="dark"]`, ce qui garantit une transition propre sans aucun rechargement de page.

4. Algorithmes utilisés

4.1. Formule de priorité dynamique

Chaque module reçoit un score de priorité calculé à chaque planification :

$$P_i = \underbrace{(\text{difficulté}_i \times \text{chapitres}_i)}_{\text{charge brute}} \times \underbrace{\left(1 + \frac{1}{\ln(\Delta t_i + 2)}\right)}_{\text{multiplicateur d'urgence}} \tag{1}$$

où Δt_i est le nombre de jours restants avant l'examen du module i.

- **Charge brute :** représente l'effort total requis.
- **Multiplicateur d'urgence :** augmente quand Δt diminue, de façon amortie grâce au logarithme pour éviter une explosion du score.

Exemple numérique concret :

Module	Difficulté	Chapitres	Δt	Charge brute	Score P
Analyse	4	6	5	24	$24 \times 1.51 \approx$ **36.3**
Algorithmes	3	5	20	15	$15 \times 1.32 \approx$ **19.8**

4.2. Entrelacement des sessions

Pour améliorer la rétention mémorielle, si un module i a déjà reçu k sessions dans la journée, son score est divisé par 2^k :

$$P_i^{\text{entrelacé}} = \frac{P_i}{2^k} \tag{2}$$

Fonctionnement

Illustration : deux modules A et B, capacité = 4 sessions/jour

Créneau	Score A	Score B	Élu	k (élu)
1	36.3	19.8	A	$0 \to 1$
2	$36.3/2 = 18.2$	19.8	B	$0 \to 1$
3	18.2	$19.8/2 = 9.9$	A	$1 \to 2$
4	$36.3/4 = 9.1$	9.9	B	$1 \to 2$

Résultat final : **A, B, A, B** — une alternance naturelle même si A est bien plus prioritaire que B.

4.3. Algorithme glouton de distribution

L'algorithme remplit les jours un par un, du premier jour disponible jusqu'à la dernière date d'examen.

4.3.1. Capacités selon le type de jour

Type de jour	Capacité	Paramètre	Condition
Soir en semaine	Variable	`heuresSoir`	Lundi au vendredi, ni férié, ni bloqué
Week-end	Variable	`heuresWeekend`	Samedi ou dimanche
Jour férié	= Week-end	`heuresWeekend`	Détecté via Nager.Date
Jour libre	= Week-end	`heuresWeekend`	Déclaré manuellement
Jour bloqué	0	–	Bloqué dans le wizard

```
joursFuturs.forEach(jour => {
  if (jour.type === 'off') return;

  let capaciteUtilisee = 0;
  const modulesDuJour = new Map();

  while (capaciteUtilisee < jour.heuresDispo) {
    const candidats = travailRestant
      .filter(m => m.sessionsRestantes > 0 && m.dateExam > jour.date)
      .map(m => {
        const k = modulesDuJour.get(m.id) || 0;
```

```
        const scoreBase = this.calculerScore(m, jour.date);
        return { ...m, scoreDynamique: scoreBase / Math.pow(2, k) };
      })
      .sort((a, b) =>
        b.scoreDynamique - a.scoreDynamique ||
        (b.chapitres || 0) - (a.chapitres || 0) ||
        a.nom.localeCompare(b.nom)
      );

    if (candidats.length === 0) break;

    const cible = candidats[0];
    jour.sessions.push({ moduleId: cible.id, faite: false });
    cible.sessionsRestantes--;
    modulesDuJour.set(cible.id, (modulesDuJour.get(cible.id) || 0) +
       1);
    capaciteUtilisee++;
  }
});
```

Listing 1 – Extrait de l'algorithme glouton (core/planning.js)

4.4. Respect du passé et recalcul du futur

À chaque recalcul, seuls les jours futurs sont modifiés. Les jours déjà passés sont conservés tels quels, ce qui préserve l'historique, évite de décourager l'étudiant, et garantit la cohérence des statistiques.

```
// Conservation de tout ce qui est <= aujourd'hui
const base = planActuel.filter(j => j.date <= today);

// G n ration uniquement des jours futurs
const debutFutur = hasToday ? lendemain : max(dateDebut, today);
const joursFuturs = this.genererJours(debutFutur, modules, config,
   joursFeeries);

return { plan: [...base, ...joursFuturs], backlog };
```

Listing 2 – Préservation du passé (core/planning.js)

4.5. Gestion du backlog et transparence

Après la distribution, s'il reste des sessions impossibles à placer, elles sont ajoutées au backlog et une alerte est affichée.

Point d'attention

Exemple de situation de backlog :
Un étudiant a un examen dans 3 jours avec 10 chapitres à réviser, mais sa capacité est de 2 sessions/soir. Il ne peut placer que $3 \times 2 = 6$ sessions maximum. Les 4 sessions restantes entrent dans le backlog, et un message apparaît : *« Surcharge : 4 sessions ne tiennent plus dans le planning. »*

5. Architecture technique

5.1. Technologies utilisées

Technologie	Version	Usage dans le projet
HTML5	–	Structure des composants, formulaires, SVG
CSS3	–	Design system, animations, responsive, mode sombre
JavaScript	ES6+	Logique métier, algorithmes, gestion de l'état
localStorage	–	Persistance des données côté client
API Nager.Date	v3	Récupération des jours fériés par pays
Web Audio API	–	Sons du minuteur Pomodoro
Google Fonts	–	Bricolage Grotesque + Plus Jakarta Sans
canvas-confetti	1.6.0	Animation de confettis à la fin d'une journée

5.2. Structure du projet

```
RevisionFlow/
|-- landing/
|   |-- index.html          # Page d'accueil
|   |-- landing.css
|   `-- landing.js          # Parallaxe, animations, compteurs
|
|-- core/                   # Modules partag s
|   |-- variables.css       # Design system complet
|   |-- state.js            # Source de verite unique (pattern Store)
|   |-- planning.js         # Algorithme de priorite et distribution
|   |-- storage.js          # Persistance localStorage + import/
    export
|   |-- ui.js               # Toasts, animations, modales
|   `-- router.js           # Navigation sans rechargement
|
`-- components/
    |-- wizard/             # Assistant de creation (3 etapes)
    |-- dashboard/          # Tableau de bord principal
    |-- filter/             # Filtres combinables
    |-- pomodoro/           # Minuteur Pomodoro
    `-- history/            # Historique des plannings
```

Listing 3 – Arborescence du projet

5.3. Gestion de l'état — pattern Store

L'état de l'application est géré par un seul objet `State`, source de vérité unique similaire à Redux mais en JavaScript pur.

```
const State = (() => {
  let _data = { ...initial };
  let _subscribers = [];

```

```
  function _sauvegarder() {
    _recalculerStats();
    Storage.sauvegarder(_data);
    _subscribers.forEach(fn => fn(deepCopy(_data)));
  }

  return {
    get()           { return deepCopy(_data); },
    update(changes) { _data = deepMerge(_data, changes); _sauvegarder
        (); },
    subscribe(fn)   { _subscribers.push(fn); },
    planifier()     { /* Lance Planning.generer() */ }
  };
})();
```

Listing 4 – Structure de State.js

5.4. Design system

Toute la charte graphique est centralisée dans `variables.css`. Le mode sombre est activé par un simple attribut `data-theme="dark"` sur la balise `<html>`, sans rechargement.

6. Défis techniques rencontrés

6.1. Préserver le passé tout en recalculant le futur

Défi technique

Problème : à chaque action, tout le planning doit être recalculé, mais les jours passés ne doivent jamais être modifiés.
Solution : avant chaque recalcul, le planning est divisé en deux parties : les jours $\leq$ aujourd'hui (conservés intacts) et les jours futurs (régénérés). Les sessions non validées d'aujourd'hui sont soustraites du volume restant pour éviter les doublons.

6.2. Synchronisation Pomodoro et planning

Défi technique

Problème : le minuteur tourne en continu. À sa fin, il doit proposer de valider la bonne session dans le planning, sans couplage direct.
Solution : à la fin d'un Pomodoro, le composant interroge `State.get()` pour trouver la première session non validée du jour, puis ouvre une modale de confirmation.

6.3. Tri déterministe de l'algorithme

Défi technique

Problème : deux modules peuvent avoir le même score. Sans tri stable, le planning change à chaque recalcul.
Solution : tri sur trois niveaux successifs — score décroissant, puis nombre de chapitres décroissant, puis ordre alphabétique. Ce dernier critère garantit toujours un résultat unique et reproductible.

6.4. Migration des données sauvegardées

Défi technique

Problème : si la structure de l'état change entre deux versions, les anciennes sauvegardes peuvent être incomplètes.
Solution : au chargement, l'état sauvegardé est fusionné avec l'état initial via un `deepMerge`. Les nouvelles propriétés reçoivent leur valeur par défaut automatiquement.

7. Interface utilisateur et design

7.1. Philosophie de design

Le design suit trois principes :

1. **Clarté :** chaque information importante est visible immédiatement.
2. **Réactivité :** les animations confirment visuellement chaque action.
3. **Cohérence :** toutes les interfaces partagent le même design system. Un bouton vert signifie toujours « valider », un orange signifie « reporter ».

7.2. Typographie

La typographie utilise **Bricolage Grotesque** pour tous les titres et chiffres (expressif et moderne) et **Plus Jakarta Sans** pour le corps du texte (lisibilité optimale pour les longues listes).

7.3. Responsive et mobile

Sur mobile (largeur ≤ 768 px) :

— La barre latérale se masque et s'ouvre via un bouton hamburger.
— La grille de statistiques passe en disposition 2×2.
— Les formulaires s'affichent en colonne unique.
— Le compte à rebours de l'examen est affiché dans la barre supérieure mobile.

8. Perspectives d'amélioration

8.1. Améliorations fonctionnelles

— **Synchronisation cloud :** accéder au planning depuis plusieurs appareils.
— **Notifications push :** rappels automatiques via Service Worker, même quand l'application n'est pas ouverte.
— **Analyse avancée :** graphiques d'évolution de la vélocité et du streak sur plusieurs semaines.
— **Import d'emploi du temps :** détecter automatiquement les créneaux libres depuis un fichier iCal.
— **Mode révision rapide :** afficher les notes par module sous forme de flashcards.

8.2. Améliorations techniques

— **Progressive Web App (PWA) :** installation sur mobile, fonctionnement hors connexion.
— **Tests unitaires :** automatiser les tests des fonctions de calcul avec Vitest.
— **Internationalisation :** traduire l'interface en anglais et en arabe.
— **Accessibilité :** navigation clavier, attributs ARIA, vérification des contrastes.

8.3. Évolutions vers le mobile

Une version mobile native (React Native ou Flutter) pourrait réutiliser directement la logique métier de `planning.js` et `state.js`, qui sont des modules JavaScript purs sans dépendance au DOM.

9. Conclusion

RevisionFlow répond à un vrai besoin étudiant en combinant une interface claire, des animations motivantes, et un algorithme de planification intelligent.

Ce projet a été l'occasion d'appliquer des concepts importants du développement web moderne : la gestion centralisée de l'état sans framework, la programmation orientée module en JavaScript pur, le design responsif avec un système de variables CSS cohérent, et la conception d'un algorithme métier reproductible et déterministe.

Résultat

RevisionFlow est un assistant de révisions intelligent, réactif et transparent. Il génère un planning personnalisé basé sur un algorithme de priorité dynamique, le met à jour en temps réel à chaque action, et avertit clairement l'étudiant lorsque ses objectifs sont irréalistes — tout cela sans serveur, sans inscription, et avec les données 100% locales.